The Journal of Open Source Software

FreeStylo: An easy-to-use stylistic device detection tool for stylometry

Felix Schneider [1¶] **and Joachim Denzler** [1]

1 Computer Vision Group, Friedrich-Schiller-Universität Jena ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 27 October 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Stylistic devices are delibarately chosen linguistic expressions that are used to convey a certain meaning or effect. They are often used in literature to create a certain atmosphere or to convey a certain message. Due to this matter, the detection of stylistic devices in text is an important task in stylometry,the study of linguistic style. Often, finding these stylistic devices is a tedious and costly process that involves close reading of the texts, ideally by multiple experts. This is extra costly especially if researchers aim to statistically analyze the usage of stylistic devices across various texts.

To improve this state, this package provides an easy-to-use command-line interface for detecting stylistic devices in text. The tool can be configured with a simple configuration file. It is designed to be usable by both experts and non-experts in programming. For those proficient in python, this package also provides a library with a collection of classes to detect stylistic devices in text, together with customizable text preprocessing (tokenizing, POS-tagging, etc.).

The command-line tool is to be used on plain text files. It pre-processes the text, detects the stylistic devices specified in the configuration file, and writes the annotations to a JSON file. The classes contained in the library can be used to work with the text in a more flexible way, e.g. by using different pre-processing methods or already pre-processed text. The resulting annotations can be saved in a JSON file or directly used as a data structure in python. Additionally the library is easily extendable with your custom stylistic device detectors.

Statement of need

Stylotool is a package that provides a collection of approaches to detect stylistic devices in text. While there exists a great variety of NLP libraries like nltk (Bird & Loper, 2004), spaCy (Honnibal et al., 2020), or cltk (Johnson et al., 2021) and command-line tools like CWB (Evert & Hardie, 2011), or UCS (Evert, 2005) for the processing and low level analysis of text, there is a lack of tools that are specifically designed for the detection of stylistic devices. Information about the usage of stylistic devices is important for many branches of stylometry, especially for the analysis of literary texts. This package aims to fill this gap by providing an easy-to-use tool and library for the detection of those stylistic devices. Due to its simple and easily configurable command-line interface, the tool itself is geared not only to people with programming knowledge, but also to literary scholars that use distant reading methods in their research. The software contained in this package is designed to be used either as a library, usable in other python programs, or as a stand-alone command-line tool.

Design and Supported Stylistic Devices

The package contains a collection of approaches to detect stylistic devices in text. By default, the preprocessing is done by spaCy(Honnibal et al., 2020) or cltk(Johnson et al., 2021). The following stylistic devices are currently supported:

Chiasmus

This package includes the current state-of-the-art approach by Schneider et al. (2021) to detect chiasmi in text. A chiasmus is a rhetorical device, that consists of two parallel phrases, where the second phrase is a semantically related inversion of the first phrase. For example, the phrase "Hard is the task, the samples are few" is a phrase that emphasizes the problem of missing examples with the oppositional posing of the words "hard" and "few".

The chiasmus detector contained in this package has been trained using the dataset published by Schneider et al. (2023). It works for English, German, and Middle High German. It has been trained with word vectors provided by the German 'de_core_news_lg' model by spaCy (Honnibal et al., 2020) However, since the model does not use the word vectors directly, but only their cosine similarity, it can be used with any word vectors, as long as they provide a vector for each token in the text.

The chiasmus detector needs some special lists to function properly.

- denylist: a list of Part of Speech (POS)-tags that are not allowed to be part of a chiasmus
- allowlist: a list of POS-tags that are allowed to be part of a chiasmus. Be careful: if such a list is given, all other POS-tags are not allowed to be part of a chiasmus.
- neglist: a list of negations in the target language.
- conjlist: a list of conjunctions in the target language.

For English, German, and Middle High German, defaults for the lists are provided in the package. However, you can provide your own lists, if you want to use the chiasmus detector for a different language or if you want to use a different set of e.g. POS-tags.

Metaphor

The metaphor detection approach in this package has specifically been developed for the low-resource language Middle High German, but can also be applied to more common high-resource language. Specifically, adjective-noun metaphors like "thirsty car" are detected using a machine-learning based rating model. The detector is based on the publication by Schneider et al. (2022).

Currently, the metaphor detector is available for English and Middle High German. The word vectors are expected to be generated by the spaCy model 'en_core_web_lg' for English and by the provided word vector FastText model for Middle High German.

Alliteration and Alliterative Verse

The package further contains a detectorfor both alliteration and alliterative verse. Alliteration comprise phrases, where the initial letters of words are the same. Since alliteration is a simple stylistic device, the detector is based on a simple rule-based approach that orders all alliterations in the given text by the number of words that are alliterated in the phrase. Additionally, the detector can also find alliterative verses, which can contain some words additional in between the words with the same initial letter. An example for alliterative verse would be *"Pondering on the pending paper, I programmed the python package."* The user can specify the maximum number of words that are allowed to be in between the alliterated words, as well as words and POS-tags that do not count towards the non-alliterated words. Since for example spaCy also tags punctuation and newlines, the user can specify those to be excluded from the alliteration.

Epiphora

An epiphora is a rhetorical device, that consists of multiple parallel phrases, where the last word of each phrase is the same. For example, *"I thought of the paper, I wrote the paper, I published the paper"* is an epiphora that emphasizes the importance of the paper. The way this detector works is by splitting the text into sentences, and then those sentences into phrases. The detector searches for adjacent phrases that end with the same word. Those phrase collections are then sorted by the number of phrases in the collection.

Polysyndeton

A polysyndeton is a rhetorical device, that consists of multiple parallel phrases, where each phrase is connected by a conjunction. For example, *"I thought of the paper, and then I started writing it, and then I published it, and then I received a lot of citations"* is a polysyndeton that, in a broader context with a slower feel to it, emphasizes the the process of writing and publishing a paper. The detector works by getting a list of all conjunctions, or by getting the POS-tag of conjunctions, or by getting both, and then splitting the text into sentences, and then counting the conjunctions in each sentence, and then sorting the sentences by number of conjunctions.

Usage

The package can be used both as a library and as a stand-alone command-line tool. Both from the library and from the command-line tool, the results can be saved in a JSON file. This json file will contain the complete tokenized text. When using the functions from the library, the result will be a python container with a similar structure to the JSON file.

Standalone Tool

The standalone version can be configured using a simple JSON configuration file. The file should specify the language of the text and the stylistic devices to detect. The following is an example configuration file:

```
{
    "language": "de",
    "annotations": {
        "chiasmus": {
            "window_size": 30,
            "allowlist": ["NOUN", "VERB", "ADJ", "ADV"],
            "denylist": [],
            "model": "/chiasmus_de.torch"
        }
    }
}
```

You can then run the tool using the following command:

```
stylotool --config config.json --input input.txt --output output.json
```

This will read the text from the file `input.txt`, preprocess (tokenize, POS-tag, etc.) the text, detect the stylistic devices specified in the configuration file, and write the results to the file `output.json`.

Library

The library comprises a collection of functions to detect the stylistic devices, as well as preprocessing based on spaCy. Should the user want to use different preprocessing or use the

DRAFT

package with a different language than the supported ones, a TextObject can be created and filled with the needed manually computed contents. The stylistic device detectors can then be applied to the TextObject.

A typical example would look like this:

```
import numpy as np

from stylotool import TextObject as to
from stylotool import TextPreprocessor as tp
from stylotool import ChiasmusAnnotation as ca
from stylotool import MetaphorAnnotation as ma

# first, create a TextObject from the raw text
text = to.TextObject(
        textfile = "example_textfile.txt",
        language="en")

# create a TextPreprocessor object and process the text
# this does the tokenizing, lemmatizing, POS-tagging, etc.
preprocessor = tp.TextPreprocessor(language="en")
preprocessor.process_text(text)

# you can also use a different preprocessing of your choice
# without the TextPreprocessor object
# just fill the TextObject with the needed contents
# those could be provided e.g. by spaCy, nltk, cltk,
# or any other method of your choice

# many digital corpora are already tokenized and POS-tagged
# they may come in various formats, such as TEI XML, CoNLL, etc.
# if you have a text in those formats, you can fill the TextObject
# with the needed contents
# you can then fill the missing values in the TextObject
# with e.g. word vectors or other features created with a method of your choice.

# you can now add various annotations to the text object
# here, we add a chiasmus annotation
chiasmus = ca.ChiasmusAnnotation(
        text=text)
chiasmus.allowlist = ["NOUN", "VERB", "ADJ", "ADV"]
chiasmus.find_candidates()
chiasmus.load_classification_model("chiasmus_model.pkl")
chiasmus.score_candidates()

# here, we add a metaphor annotation
metaphor = ma.MetaphorAnnotation(
        text=text)
metaphor.find_candidates()
metaphor.load_model("metaphor_model.pkl")
metaphor.score_candidates()

# finally, save the annotated text to a json file
text.serialize("annotated_text.json")
```

DRAFT

Create your own detectors

The package is designed to be easily extendable with your own stylistic device detectors. The `src` folder contains example scripts that show how you can retrain the models for the existing chiasmus and metaphor detectors. You can also create your own stylistic device detectors by referring to the existing ones. Especially the Alliteration Detector provides a very simple example that can be used as a template for your own detectors. If you create your own detecors, pull requests are very welcome!

References

Bird, S., & Loper, E. (2004). NLTK: The natural language toolkit. *Proceedings of the ACL Interactive Poster and Demonstration Sessions*, 214–217. https://aclanthology.org/P04-3031

Evert, S. (2005). Empirical research on association measures: The UCS toolkit. *Phraseology 2005 Conference*.

Evert, S., & Hardie, A. (2011). Twenty-first century corpus workbench: Updating a query architecture for the new millennium. *Proceedings of the Corpus Linguistics 2011 Conference, University of Birmingham, UK*.

Honnibal, M., Montani, I., Van Landeghem, S., & Boyd, A. (2020). *spaCy: Industrial-strength Natural Language Processing in Python*. Zenodo. https://doi.org/10.5281/zenodo.1212303

Johnson, K. P., Burns, P. J., Stewart, J., Cook, T., Besnier, C., & Mattingly, W. J. B. (2021). The Classical Language Toolkit: An NLP framework for pre-modern languages. *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*, 20–29. https://doi.org/10.18653/v1/2021.acl-demo.3

Schneider, F., Brandes, P., Barz, B., Marshall, S., & Denzler, J. (2021). Data-driven detection of general chiasmi using lexical and semantic features. *SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature*, 96–100. https://doi.org/10.18653/v1/2021.latechclfl-1.11

Schneider, F., Sickert, S., Brandes, P., Marshall, S., & Denzler, J. (2023). Hard is the task, the samples are few: A german chiasmus dataset. *Language Technology Conference: Human Language Technologies as a Challenge for Computer Science and Linguistics (LTC)*, 255–260. https://doi.org/10.14746/amup.9788323241775

Schneider, F., Sickert, S., Brandes, P., Marshall, S., & Denzler, J. (2022). Metaphor detection for low resource languages: From zero-shot to few-shot learning in middle high german. *LREC Workshop on Multiword Expression (LREC-WS)*, 75–80.

DRAFT